1) Give an explicit update rule for the knowledge bases for Type A Bot and Type B Bot, and justify that your formulas are correct.

The generic LocatorBot begins with a complete map of the ship, but no knowledge of the LostBot's location. The ship contains both closed and open cells, with closed cells representing walls or other barriers, and open cells representing rooms. Every open cell, excluding the LocatorBot's starting position, is initially a possible location for the LostBot, assuming the two bots cannot begin in the same cell. The LocatorBot attempts to locate the LostBot using a detector that determines whether the LostBot occupies its current cell or provides information about its location based on the type of detector being used. Type A represents this uncertainty as a set of locations that remain logically possible, since its detector produces deterministic readings. Type B assigns a probability to each possible location, since its detector produces probabilistic readings whose likelihoods depend on the distance between the two bots.

For Type A, let S be the set of currently possible LostBot locations, let ‘a’ be the LocatorBot's position, and let R(a, k) contain every open cell whose row and column differ from a by at most k cells. This is a square detector region, and intervening walls do not block it. After running the detector, the bot applies one of the following rules:

> PING: $S_{new} = (S \cap R(a, k)) \setminus a$. (The new set of possible LostBot locations consists of only those cells that are both in the current set of possible locations and within the detector's range, excluding the cell the LocatorBot currently occupies.)
>
> NO_PING: $S_{new} = S \setminus R(a, k)$. (The new set of possible LostBot locations consists of all cells in the current set of possible locations, excluding every cell within the detector's range.)

A PING retains only cells in the square, while a NO_PING eliminates the entire square. If the bot occupies the LostBot's cell, the detector instead returns FOUND, which is why ‘a’ is excluded regardless of the reading. To prove these update rules are correct, assume that an update eliminates the LostBot's actual location. If the detector returns PING, the update eliminates all locations outside the detection region. This would imply that the LostBot was outside the region, contradicting the PING observation. If the detector returns NO_PING, the update eliminates all locations inside the detection region. This would imply that the LostBot was inside the region, contradicting the NO_PING observation. Therefore, neither update can eliminate the LostBot's actual location. Since the LostBot remains stationary, previously eliminated locations cannot become valid again, and the set of possible locations can only shrink. Furthermore, each update removes every candidate inconsistent with the observed reading, ensuring that the resulting set contains exactly the locations still logically possible.

For Type B, let P(b) represent the current probability that the LostBot occupies a particular cell 'b'. If the ship contains N open cells, and the LocatorBot and LostBot cannot begin

in the same cell (I assume, as the project did not specify), the LocatorBot's starting cell receives a probability of zero, while every other open cell initially receives a probability of 1/(N−1). This is because the LocatorBot has no reason to believe that the LostBot is more likely to occupy any one of the remaining open cells than another. Unlike Type A, which can eliminate entire regions with certainty, Type B must adjust the probabilities of possible locations based on how likely each location is to produce the observed detector reading. Let 'a' represent the LocatorBot's current position, 'b' represent a possible LostBot location, 'd' represent the Euclidean distance between these two positions, and 'α' represent the detector's sensitivity parameter. The probability of receiving either detector reading is calculated using the following formulas:

> $P(\text{PING} \mid a, b) = \exp[-\alpha(d - 1)]$. (The probability that the detector returns PING when the LostBot is at cell 'b'. This probability decreases as the distance between the two bots increases)
>
> $P(\text{NO_PING} \mid a, b) = 1 - \exp[-\alpha(d - 1)]$. ((The probability that the detector returns NO_PING when the LostBot is at cell 'b'. This probability increases as the distance between the two bots increases)
>
> After receiving a detector reading, Type B updates its knowledge base using Bayes' theorem. Let 'O' represent the observed reading, either PING or NO_PING. The updated probability of the LostBot occupying any particular cell 'b' is calculated as follows:
>
> $P_{new}(b) = P(O \mid a,b) \times P_{old}(b) / \Sigma_{c \neq a} [P(O \mid a,c) \times P_{old}(c)]$ (The new probability of the LostBot occupying cell 'b' is calculated by multiplying its previous probability by the likelihood of receiving the observed detector reading if the LostBot were at that cell. This result is then divided by the sum of the corresponding products for every remaining candidate cell, excluding the LocatorBot's current position.)

The LocatorBot's current position is excluded because occupying the same cell as the LostBot would have returned FOUND. Dividing by the sum of all remaining probabilities ensures that the updated probabilities add up to one. A PING favors nearby cells over distant cells because nearby locations have a higher likelihood of producing that observation. Conversely, a NO_PING favors distant cells because they have a higher likelihood of producing no signal. A NO_PING also eliminates any candidate exactly one orthogonal step away, since a LostBot at that distance would have produced a PING with 100% certainty. To prove this update rule is correct, assume for the sake of contradiction that it calculates an incorrect probability for some candidate cell 'b' after receiving an observation 'O'. By the definition of conditional probability, the probability that the LostBot occupies cell 'b' after observing 'O' must equal the probability that both events occur, divided by the total probability of observing 'O'. The probability of both events occurring is $P_{old}(b) \times P(O \mid a, b)$, while the total probability of the observation is the sum of these products across all remaining candidate cells. Therefore, the updated probability must equal the result of our formula, contradicting the assumption that the

formula was incorrect. This proves that the update correctly applies the information provided by the detector, assuming the initial probabilities and detector likelihoods are correct.

2) Outline your strategy for using the Type A and Type B detectors. Be clear about any decisions you're making, any justifications you are using for your design choices. Why is your strategy a good idea, and why should it be better than the baseline choices? Try to utilize the ideas of the course to justify yourself (theoretically and/or practically). Be sure to specify how each bot strategy utilizes its current state of knowledge.

The baseline Type A strategy detects, updates its possible-location set, and moves to the nearest cell that might contain the LostBot. It uses breadth-first search (BFS) to find the nearest candidate by walking distance, resolving equally close candidates randomly. The weakness of this strategy is that the nearest candidate is not necessarily the best location to run the detector. Following a NO_PING, the baseline must travel outside the previously eliminated region, but its next detection square often overlaps considerably with cells already ruled out, reducing the number of unresolved candidates covered by the detector. Additionally, the baseline only considers remaining possible LostBot locations as destinations, ignoring cells that have already been eliminated, even when those cells could serve as useful detection points. Consequently, it can travel a large distance to investigate a candidate when a closer, previously eliminated cell could provide equally or more useful information. Essentially, the baseline always selects its next destination without evaluating the expected information gained from a detection there.

My improved Type A strategy treats the next destination as an information-gathering decision. After each reading, it evaluates every reachable open cell, excluding its current position, as a potential detector position, including cells that have already been ruled out as LostBot locations. For each position, it counts how many currently viable target locations fall within the detector square and calculates how many possibilities would be expected to remain after PING, NO_PING, or FOUND. Because the initial target distribution is uniform and Type A readings are deterministic, all candidates remaining in its knowledge base are equally likely, so this expected reduction can be calculated from the candidate counts alone.

More specifically, if 'n' candidates remain, 'r' are inside the proposed detector square, and 's' equals 1 if the proposed detector cell itself could contain the LostBot, or 0 otherwise, a positive reading leaves r − s possibilities, a negative reading leaves n − r, and FOUND ends the search with none. Weighting these outcomes by their probabilities gives the expected number of candidates remaining:

$$\text{Expected remaining} = [(r - s)^2 + (n - r)^2] / n$$

The bot subtracts this quantity from n to estimate the expected number of candidates the detection would eliminate. It then divides that benefit by the number of BFS movements

required to reach the proposed cell plus the detector cost of 0.5. The chosen destination has the highest expected elimination per unit of immediate cost. A score tie is resolved in favor of the shorter route, with any remaining ties broken randomly. Once the bot reaches the destination and detects, it updates its knowledge base and repeats the evaluation.

The baseline Type B strategy also uses BFS, but it identifies the single cell with the highest posterior probability and travels there regardless of how expensive the journey is. Only when probabilities tie does the baseline use walking distance to choose between candidates, breaking any remaining ties randomly. This decision rule maximizes the immediate probability of finding the LostBot at the destination, but it can be wasteful when the most likely cell is far away and only marginally more probable than a nearby alternative. For example, suppose one candidate has a probability of 0.361 and requires 40 movements to reach, while another has a probability of 0.358 and requires only 5 movements. The baseline will always select the first candidate, despite its significantly higher investigation cost for a relatively small increase in discovery probability.

My improved Type B bot keeps exactly the same Bayesian update rule as the baseline, its advantage instead comes from destination selection. After each reading, the bot runs BFS once to determine the shortest walking distance to all reachable cells. For each possible LostBot location, it divides the current probability assigned to that cell by the travel distance required to reach it plus the detector cost of 0.5, effectively weighting each candidate's probability against its investigation cost. It selects the candidate with the highest probability per unit of investigation cost, breaking score ties randomly. It then follows the shortest path to that cell, runs its detector, incorporates the observation into its probability distribution, and repeats. This directly addresses the baseline's tendency to cross the ship for small increases in posterior probability. In the previous example, the improved bot would prefer the closer candidate because its probability relative to investigation cost is considerably higher.

Both improved strategies use greedy heuristics, selecting destinations based on their immediate expected benefit. Type A attempts to maximize the expected number of candidates eliminated per cost unit, while Type B attempts to maximize the probability of immediate discovery per cost unit. The improved Type B strategy does not explicitly account for the information a detector reading could provide about surrounding locations, while the improved Type A strategy does not consider how its current decision might affect the efficiency of future detections. Nevertheless, both strategies directly address limitations of their respective baselines.

3) Compare Baseline Type A strategy to your Type A strategy, graphing the discovery cost as a function of k. Note you'll need to repeat multiple experiments to collect sufficient data here. Where is the most interesting k-regime? Note, you should aim to do these experiments on as large a ship as you can, within the constraints of time allowed.

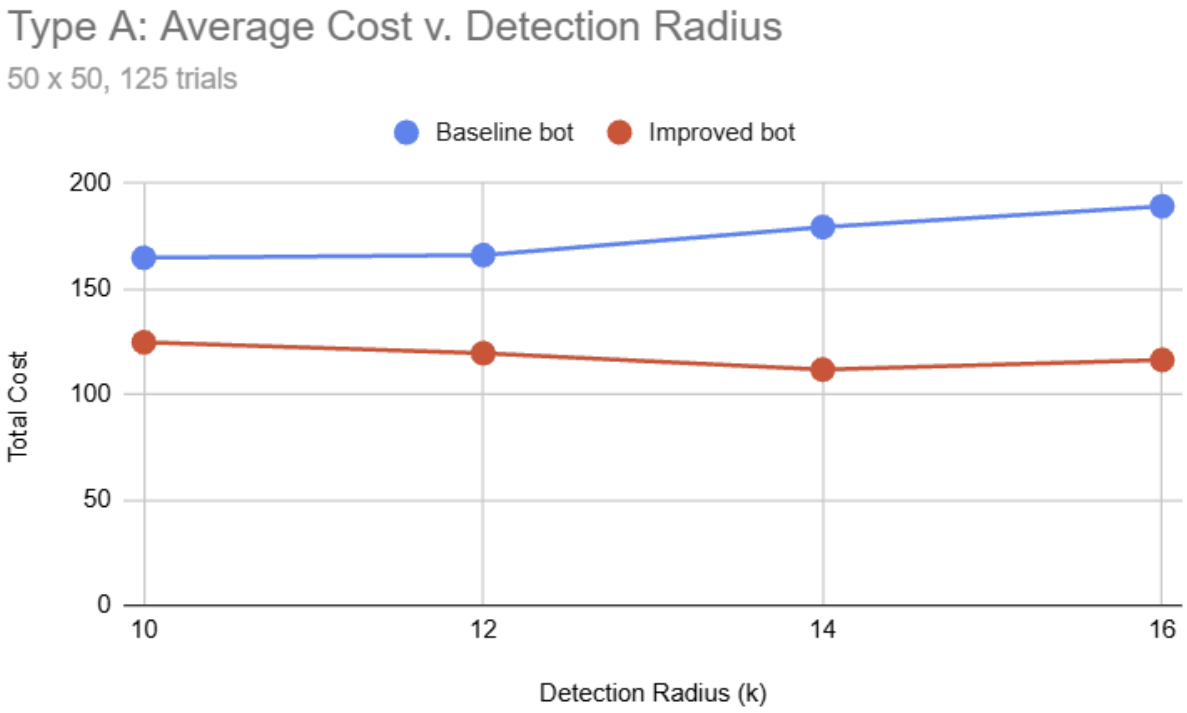
Per the project specification, cost is defined as the number of movements plus half the number of detector actions. To control for environmental difficulty, every baseline and improved comparison used the same generated ship layout, LocatorBot starting cell, and LostBot position within each trial. I first ran exploratory 20×20 experiments at various radii to identify which were informative, then moved to 50×50 ships for the main analysis. At 50×50, I conducted three experiments with 15, 30, and 80 trials per tested setting, respectively. These experiments produced 125 independent paired trials for each of k = 10, 12, 14, and 16. Other radii were explored throughout these experiments, but these four values were kept consistent to allow for a more reliable comparison.

Setting	Standard-cost comparisons	Free-movement comparisons
Main ship size	50 × 50	50 × 50
Main trial batches	15 + 30 + 80	20 + 40 + 100
Movement cost	1.0 per move	0.0 per move
Detection cost	0.5 per activation	0.5 per activation
Search environment	Random connected open cells	Same generation method
Comparison protocol	Paired ship/start/target per trial	Paired ship/start/target per trial
Primary outcome	Moves + 0.5 × detections	0.5 × detections

The preliminary sweep established why the detector radius must be tested across different scales. When k = 0, the detector provides no information about other cells, while at k = 19 on a 20×20 ship, the detector covers the entire ship and cannot narrow down the search space beyond excluding the LocatorBot's current cell. As k moves away from these extremes, detector readings become more useful because they can eliminate larger groups of possible locations without covering the entire search area.

On 20×20 ships, the baseline's average cost was 329.90 at k = 0 and 365.55 at k = 19, compared with a substantially lower cost of 52.15 at k = 5. The difference between the two extreme values is likely attributable to random variation in the initial sweep's small sample of 10 trials, since both settings provide equally uninformative detector readings. These preliminary results suggest that the most efficient detector radius lies somewhere between the extremes..

On 50×50 ships, the most interesting region was k = 10 through k = 16. Combining all 125 trials at the shared settings, the baseline's lowest observed mean cost was 164.74 at k = 10, while the improved bot's was 111.72 at k = 14. Thus, after allowing each strategy to use its best tested radius, the improved bot reduced average discovery cost by 32.2%. To isolate the strategy's effect from the detector radius, I also compared both bots at k = 14. The baseline averaged 179.28 and the improved bot 111.72, a 37.7% reduction with the detector held constant. The improved bot was cheaper in 91 of the 125 paired trials. The mean paired saving was 67.55 cost units, with a 95% confidence interval of approximately 49.83 to 85.28 units. Since this interval is entirely positive, the results provide strong evidence that the improved strategy reduces discovery cost.



The action counts explain where the savings occurred. At k = 14, the baseline averaged 152.82 movements and 52.90 detector actions. The improved bot averaged 103.61 movements and 16.23 detections. It reduced the number of detections by approximately 69% while also taking substantially fewer steps. Its cost standard deviation was 39.44, compared with 104.58 for the baseline, indicating that its discovery cost was also considerably less variable. The final 80-trial validation independently reproduced the result at k = 14: the baseline averaged 188.27, the improved bot 115.21, and the improved bot was cheaper in 59 of the 80 matched trials.

Trial 1 of Experiment 5 (Runs 9 and 10) provides a concrete example of this improvement. Both bots began at (47, 28), the LostBot was at (16, 16), and the initial k = 14 reading returned NO_PING for both. This left 1,318 possible target locations. The baseline selected (49, 43), the closest candidate under its own rule, requiring 19 movements before the next detection. The improved bot selected (43, 28), only four movements away. That destination had already been ruled out as the LostBot's location by the first NO_PING, yet it remained an effective observation point because its detector square covered unresolved cells. The baseline's proposed reading was expected to eliminate approximately 130.88 candidates, compared with 139.69 for the improved bot's proposed reading. Once travel and detector costs were included, the improved choice provided approximately 31.04 expected eliminations per cost unit, compared with 6.71 for the baseline's choice.

Because Type A detection is deterministic, this decision also allows a retrospective comparison of the two destinations. The baseline's second detection at (49, 43) returned NO_PING, eliminating 69 of its 1,318 remaining candidates after 19 movements. Had it instead followed the improved strategy to (43, 28), its second reading would also have returned NO_PING, because the LostBot was actually at (16, 16). That reading would have eliminated 74 candidates after just four movements. The baseline therefore paid 15 additional movement units to eliminate five fewer possible locations, leaving 1,249 candidates instead of 1,244. Over the complete trial, the baseline required 167 movements and 88 detections, for a total cost of 211.00, while the improved bot required 89 movements and 16 detections, for a total cost of 97.00. Although their later decisions differed, this initial comparison demonstrates how the improved strategy can gain more information at a lower cost by considering previously eliminated cells as potential detection positions.

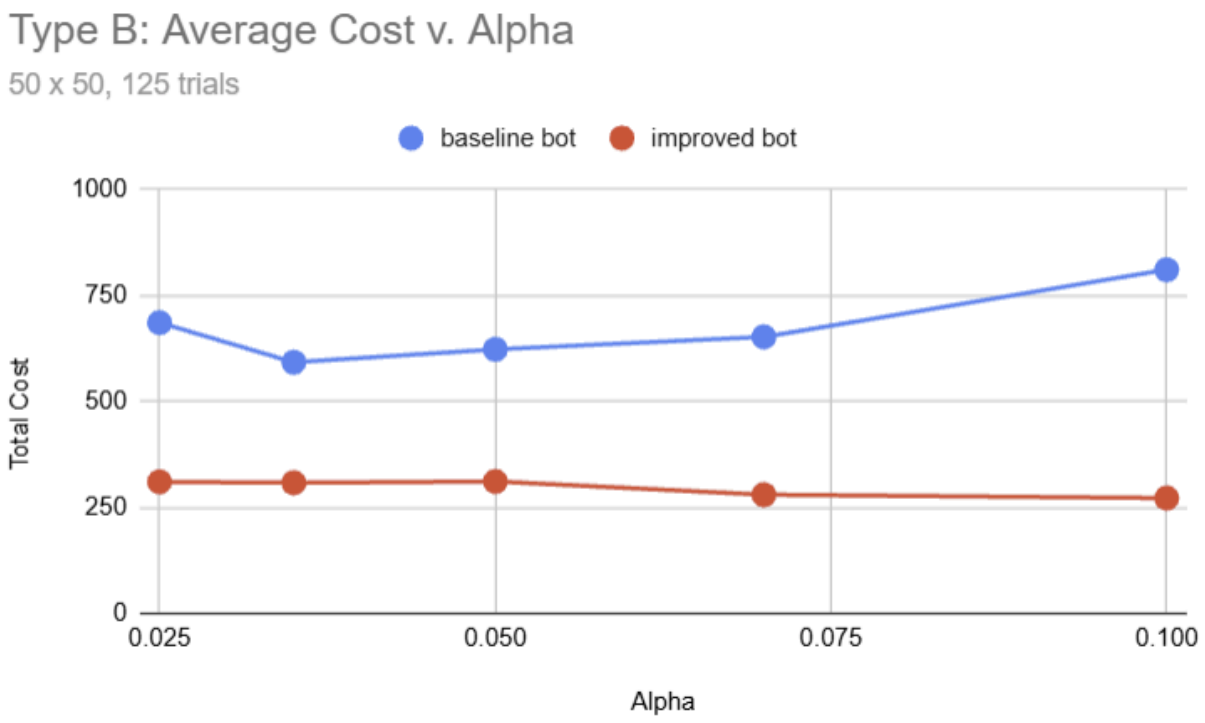
However, the improved strategy did not outperform the baseline in every trial. In Experiment 5, Trial 20 (Runs 351 and 352), both bots began at (21, 43), with the LostBot at (8, 41). Their initial PING left 391 possible locations. The baseline moved to (22, 43) and received another PING, then moved to (23, 43) and received NO_PING. This third reading reduced its remaining candidates from 378 to just 15 after only two movements. The improved bot instead selected (20, 45) and then (19, 46), receiving PING at both positions and retaining 308 candidates after five movements. The baseline ultimately found the LostBot after 19 movements and six detections, costing 22.0 units, while the improved bot required 87 movements and 12 detections, costing 93.0 units. This occurred because the baseline happened to select a detector position whose NO_PING eliminated nearly all remaining candidates, while the improved bot's chosen positions produced less favorable observations. The improved strategy maximizes expected immediate candidate elimination relative to cost, but cannot predict the actual detector outcomes or guarantee the most efficient sequence of future actions. This explains why it can perform worse on individual trials despite its lower average discovery cost.

The most efficient tested radius for my improved Type A strategy is therefore $k = 14$. Its average cost of 111.72 was the lowest observed across every radius tested on 50×50 ships, including the equally sampled alternatives $k = 12$ at 119.49 and $k = 16$ at 116.31. The radius $k = 14$ also achieved the lowest improved Type A average separately in Experiments 3, 4, and 5, making the selection consistent across independent batches of ships. However, the differences between $k = 14$ and its nearby alternatives were relatively small, so these results establish it as the most efficient tested radius rather than proving it is uniquely optimal. The baseline's most efficient tested radius was separately $k = 10$, demonstrating that the preferred detector radius also depends on the strategy used to select detection positions.

4) Compare Baseline Type B strategy to your Type B strategy, graphing the discovery cost as a function of α. Note, be very careful here about what a good range of α is! How did you make your choices?

Choosing α requires more care because it determines how rapidly the probability of receiving PING decreases with Euclidean distance. At a separation of ten cells, α = 0.025 produces a ping probability of approximately 79.9%, α = 0.10 produces 40.7%, and α = 0.35 produces 4.3%. At a separation of twenty-five cells, these probabilities become approximately 54.9%, 9.1%, and 0.02%, respectively. Very small α values produce frequent pings even when the LostBot is distant, making positive readings less useful for distinguishing between possible locations. Very large α values make pings extremely unlikely until the bot gets relatively close, leaving it with little information about which distant regions are worth investigating. The appropriate range also depends on ship size, because a 50×50 ship contains considerably larger distances between cells than a 20×20 ship.

Rather than choosing α from a single arbitrary range, I tested it in stages. The first 20×20 exploratory experiment included values from 0.01 to 1.20. A second 20×20 sweep concentrated on smaller values that appeared more promising. After increasing the ship dimension to 50, the third experiment tested values from 0.015 through 0.15. The fourth used 0.025 through 0.35 to examine both the promising lower-α region and the deterioration at higher settings. Finally, an 80-trial validation narrowed the range to 0.025, 0.035, 0.05, 0.07, and 0.10. This process allowed the preliminary results and the increased ship dimensions to guide the selection of subsequent parameters. The three 50×50 experiments provide 125 paired trials at each of these five α values.



The pooled 50×50 results show that the two strategies benefit from different parameter values. The baseline performed best at α = 0.035, averaging 592.18 discovery-cost units. The

improved bot performed best at $\alpha = 0.10$, averaging 271.97. Comparing the best tested setting of each strategy gives a 54.1% reduction in average cost. To isolate the strategy's effect from the detector parameter, I also compared both bots at $\alpha = 0.10$. The baseline averaged 810.44 and the improved bot 271.97, a reduction of 66.4%. The improved bot was cheaper in 110 of those 125 paired trials. The mean paired saving was 538.48 units, with a 95% confidence interval of approximately 438.07 to 638.88 units. Since this interval is entirely positive, the results provide strong evidence that the improved strategy reduces discovery cost.

The savings came overwhelmingly from movement rather than detector use. At $\alpha = 0.10$, the baseline averaged 791.61 movements and 37.67 detections, compared with 252.65 movements and 38.64 detections for the improved bot. The improved bot used slightly more detections on average, but the additional detector cost was negligible compared with the hundreds of movements it avoided. Its average cost was also considerably less variable, with a standard deviation of 181.63 compared with 549.21 for the baseline.

Trial 1 of Experiment 5 (Runs 17 and 18) provides an example of this improvement at $\alpha = 0.10$. As discussed in the previous question, both bots began at (47, 28), with the LostBot located at (16, 16). Both initially received NO_PING. After this, the baseline's most probable candidate was (0, 0), with a posterior probability of approximately 0.0702%. It committed to a 75-movement route to run its next detector there. The improved bot instead chose (42, 28), only five movements away, even though that cell had a lower posterior probability of approximately 0.0232%. The distant candidate was roughly three times as likely to contain the LostBot, but it required nearly fourteen times the investigation cost. The improved candidate therefore offered approximately 4.5 times the immediate discovery probability per cost unit. Over the complete trial, the baseline moved 633 times and detected 22 times, for a cost of 644.00. The improved bot moved 141 times and detected 42 times, for a cost of 162.00. Although the improved bot used 20 additional detections, it avoided 492 movements, resulting in a substantially lower total cost. Because Type B detection is stochastic, the two bots received different observations after their initial reading, so the entire cost difference cannot be attributed solely to their first destination choices. However, their identical initial observation allows us to compare those decisions directly, and the broader paired results demonstrate that the improvement is consistent across many ship environments.

The experiments also demonstrate why simply increasing α is not a solution, especially for the baseline. At $\alpha = 0.35$ on experiment 4, the baseline's mean cost climbed to 1,772.95 and the improved bot's to 360.98 across 30 trials. Because the baseline continues to prioritize whichever cell has the highest posterior probability, regardless of how far it must travel to reach it, its performance decreases sharply at larger α's.

After considering all tested values, I select $\alpha = 0.10$ as the most efficient tested parameter for the improved Type B strategy. It produced the lowest pooled 50×50 average of 271.97 across 125 trials, compared with 279.86 at $\alpha = 0.07$. However, the 80-trial slightly favored $\alpha = 0.07$,

averaging 285.84 compared with 290.95 for $\alpha = 0.10$. This difference is small relative to the variation between trials, and the pooled comparison does not establish a statistically significant advantage for either nearby value. Therefore, $\alpha = 0.10$ is the most efficient tested setting according to the combined experimental averages, but $\alpha = 0.07$ remains a similarly competitive alternative. The baseline's most efficient tested parameter was separately $\alpha = 0.035$, with an average cost of 592.18. These selections represent the lowest observed averages for the respective strategies and tested configurations.

5) At the 'most efficient k' and the 'most efficient α', which detector type is more 'efficient', and why?

I define the most efficient detector parameter as the tested value producing the lowest mean discovery cost under the assignment's cost model, where each movement costs one unit and each detection costs 0.5 units. As established in Questions 3 and 4, the most efficient tested parameters for my improved strategies were $k = 14$ for Type A, averaging 111.72 cost units, and $\alpha = 0.10$ for Type B, averaging 271.97. Type A was therefore 58.9% cheaper when each detector was used with its respective improved strategy and best tested parameter.

This comparison was based on 125 paired trials across three experiments on 50×50 ships. Both bots were tested on the same ship layout, starting position, and LostBot location within each trial, allowing their performance to be compared without differences in the environment influencing the results. Improved Type A achieved a lower discovery cost in 105 of these 125 trials, with an average paired saving of 160.24 units and a 95% confidence interval of approximately 127.90 to 192.59 units. Since this interval is entirely positive, the results provide strong evidence that Type A was more efficient under the tested conditions. The separate 80-trial validation supported this conclusion, with Type A averaging 115.21 cost units compared with Type B's 290.95, and Type A performing better in 69 of the 80 matched trials.

At their most efficient tested parameters, improved Type A averaged 103.61 movements and 16.23 detector actions, while improved Type B averaged 252.65 movements and 38.64 detector actions. Type A therefore required approximately 59% fewer movements and 58% fewer detections. Type A's ability to locate the LostBot with fewer journeys around the ship seems to provide a substantial advantage, as it makes up the bulk of cost.

I believe the primary explanation for Type A's advantage is the certainty of the information provided by its detector. A NO_PING eliminates every possible location inside the detection square, while a PING eliminates every location outside it. This allows the improved Type A strategy to evaluate how effectively each proposed detection would divide the remaining candidates and select positions that maximize expected elimination relative to investigation cost. Since the readings are deterministic, each observation establishes definite constraints on where the LostBot can be located, allowing the bot to narrow its knowledge base without physically investigating every candidate. Type B receives less definitive information because its detector

readings are probabilistic. A distant LostBot can still generate PING, while a relatively nearby LostBot can sometimes generate NO_PING. Its Bayesian updates correctly account for these likelihoods, but most observations only change the relative probabilities of possible locations rather than eliminating them outright. Consequently, Type B may require multiple observations to determine which locations are worth investigating. My improved Type B strategy reduces unnecessary movement by prioritizing candidates according to their probability of containing the LostBot relative to investigation cost. But, it does not explicitly evaluate the information that a detector reading might provide about surrounding locations, which gives Type A's information-oriented strategy an additional advantage.

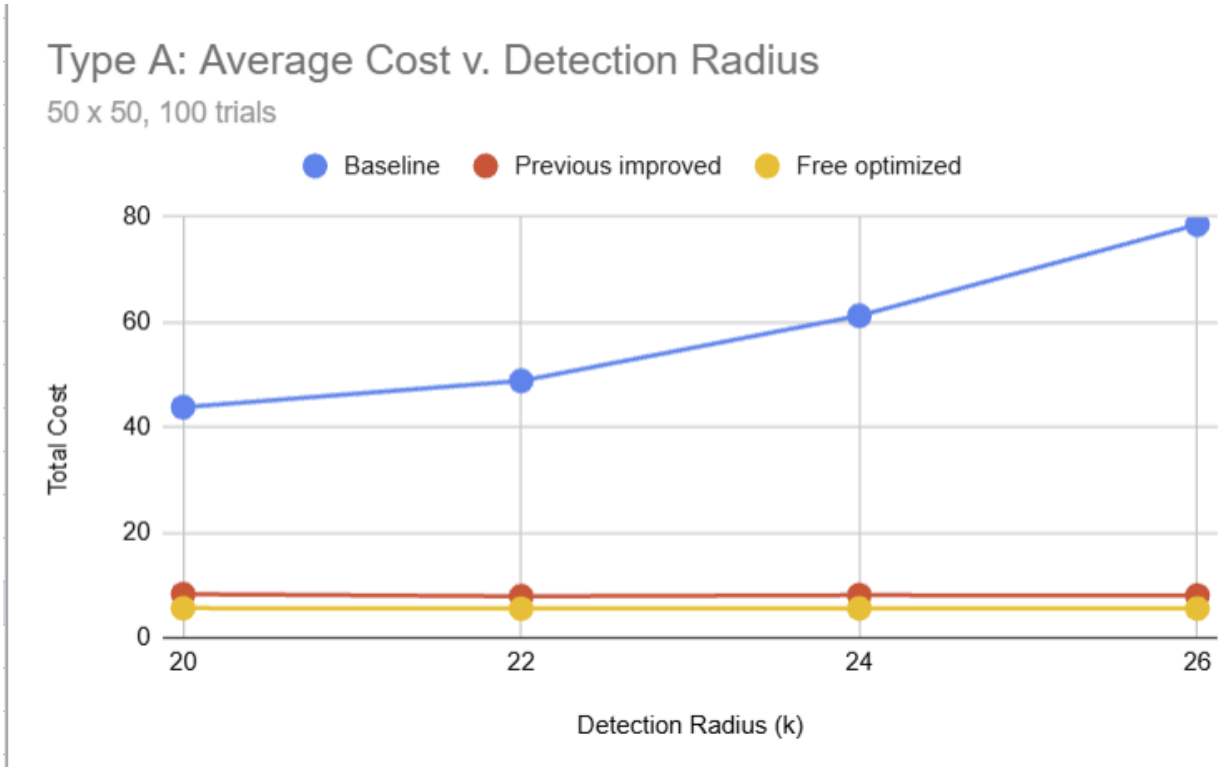
The same conclusion holds when comparing the original baseline strategies at their respective best tested parameters. Baseline Type A achieved an average cost of 164.74 at $k = 10$, while baseline Type B averaged 592.18 at $\alpha = 0.035$, making Type A approximately 72.2% cheaper. This demonstrates that Type A's advantage was also present before introducing my improved strategies.

Overall, Type A with $k = 14$ was the most efficient detector and strategy combination tested, achieving an average discovery cost of 111.72 compared with Type B's 271.97 at $\alpha = 0.10$. Its advantage can be attributed to the deterministic information provided by its detector and the improved strategy's ability to use that information when selecting detection positions. However, there isn't enough proof to say that every possible Type A algorithm would outperform every possible Type B algorithm.

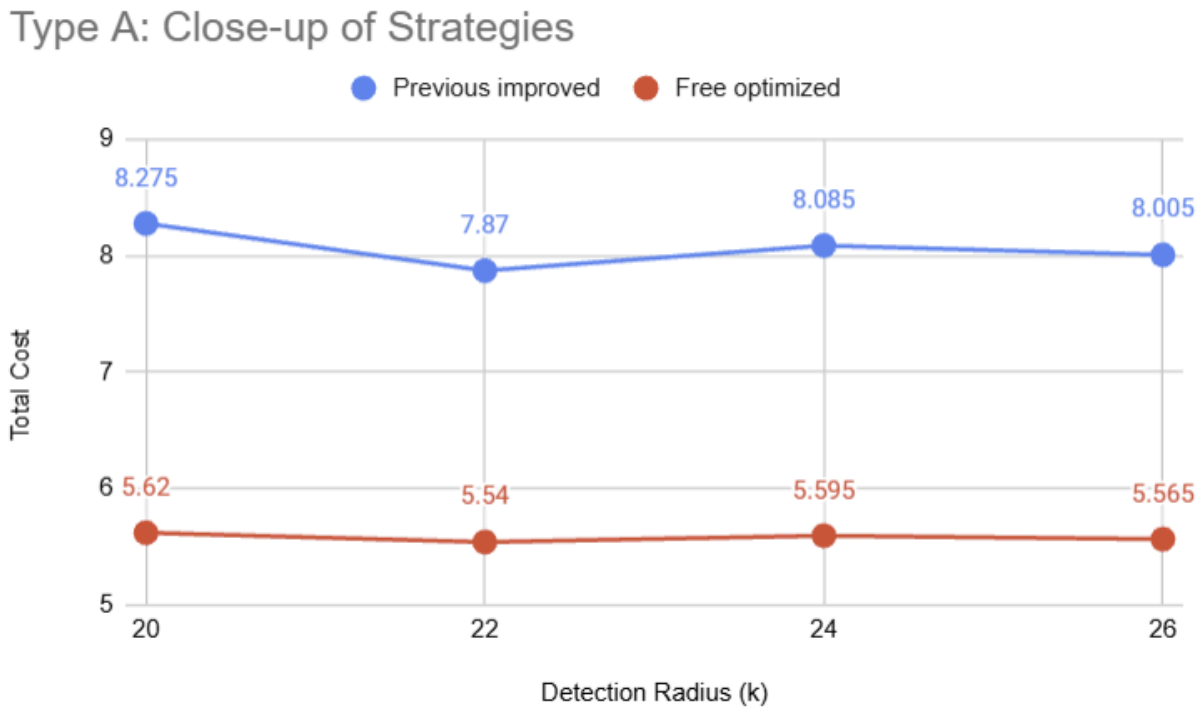
6) Suppose that movement is free, and the only thing that costs anything is running the detector. How does this change your strategies? Implement this change and compare your results to the baseline strategies and your previous strategies. Which detector is more efficient here? (Tuned optimally). Be clear in your analysis and experimentation.

Removing the cost of movement substantially changes how both improved strategies should operate. Previously, both algorithms considered the distance required to reach a detection position when selecting their next destination. With movement now costing zero, the objective becomes minimizing the number of detections required to find the LostBot. Consequently, traveling farther to obtain a more informative detector reading can be preferable to making several nearby detections. To investigate this change, I implemented two additional strategies, Type A Free and Type B Free. All six strategies were evaluated under the new cost model. Within each trial, every configuration used the same generated ship, LocatorBot starting position, and LostBot location, although Type B detector outcomes remained probabilistic. I conducted three experiments on 50×50 ships, beginning with 20 trials per configuration, increasing to 40, and concluding with 100 trials per configuration. Across these experiments, 5,280 simulations completed successfully. The following analysis primarily uses the final 100-trial experiment, allowing each strategy to be compared on the same number of environments.

For Type A, the new strategy modifies the scoring method used by my previous improved bot. Previously, it calculated the expected number of candidate locations eliminated by a detection and divided that benefit by the movement and detector costs required to reach the proposed position. With movement free, I removed the movement penalty and instead selected the reachable detector position with the greatest expected reduction in possible LostBot locations. The bot retains the same deterministic knowledge-base updates and only considers walking distance when resolving ties between equally informative destinations. This allows it to travel farther when doing so would produce a more useful detector reading, without being discouraged by a cost that no longer exists.



The results demonstrate a substantial improvement from adapting Type A's strategy to the new cost model. At k = 22, the free-optimized bot averaged 5.540 discovery-cost units, equivalent to 11.08 detections. At the same radius, the baseline averaged 48.815, while the previous improved strategy averaged 7.870. The free-optimized strategy therefore reduced discovery cost by 88.7% relative to the baseline and 29.6% relative to the previous improvement. Across the 100 paired trials, it outperformed the previous improved bot 87 times, tied four times, and performed worse nine times. The average paired saving was 2.330 cost units, with a 95% confidence interval of approximately 1.930 to 2.730 units. These results provide strong evidence that removing the unnecessary travel penalty improved detector efficiency.



The most efficient tested radius for the free-optimized Type A strategy was k = 22. At k = 20, 22, 24, and 26, its average costs were 5.620, 5.540, 5.595, and 5.565, respectively. These values are very close, indicating that the strategy performs consistently across several nearby radii. The preceding 40-trial experiment also produced a similar average of 5.525 at k = 22. Although k = 22 had the lowest observed mean in the final experiment, the differences between nearby settings were small enough that they did not establish a unique optimal radius.

A recorded trial provides a clear explanation for Type A's improvement. In the final free-movement experiment, Trial 58 (Runs 1547 and 1548), the previous improved and free-optimized bots both began at (49, 37), with the LostBot at (0, 44). Their initial readings returned NO_PING, reducing the knowledge base from 1,627 possible locations to 1,101. The previously improved bot then traveled eight cells to (42, 38), where its next detection would divide the remaining candidates into groups of 156 and 945. The free-optimized bot instead traveled 55 cells to (32, 7), where its detector would divide the candidates into groups of 552 and 549. Both readings returned NO_PING, leaving the previous improved bot with 945 possible locations and the free-optimized bot with only 549. This decision demonstrates the advantage of removing the movement penalty. The previous strategy favored a nearby detection position despite its relatively uneven division of the remaining candidates. The free-optimized strategy traveled farther to obtain an almost equal division of the search space. Over the complete trial, the previous improved bot required 25 detections and 128 movements, costing 12.5 units. The free-optimized bot required only 11 detections despite traveling 254 movements, resulting in a

cost of 5.5 units. The 14 avoided detections produced a 56% reduction.

Type B required a different approach because its probabilistic detector generally cannot eliminate entire regions with certainty. My previous improved Type B strategy prioritized candidate locations according to their probability of containing the LostBot relative to the cost of reaching them. Under free movement, this rule continues to penalize distant destinations unnecessarily. The new Type B Free strategy instead evaluates how much information a detector reading at each reachable position could provide about the LostBot's location. To accomplish this, I used a variance-based heuristic that compares the probability of receiving PING across possible LostBot locations. A detector position is considered informative when different possible LostBot locations predict substantially different probabilities of receiving PING, since the resulting observation could help distinguish between those locations. The strategy also considers the probability of immediately finding the LostBot at the proposed detector position.

Let p(b) represent the current probability that the LostBot occupies cell 'b', and let 'a' represent a proposed detection position. The scoring method uses the following values:

$f(a) = p(a)$ (the probability of immediately finding the LostBot at the proposed position.)

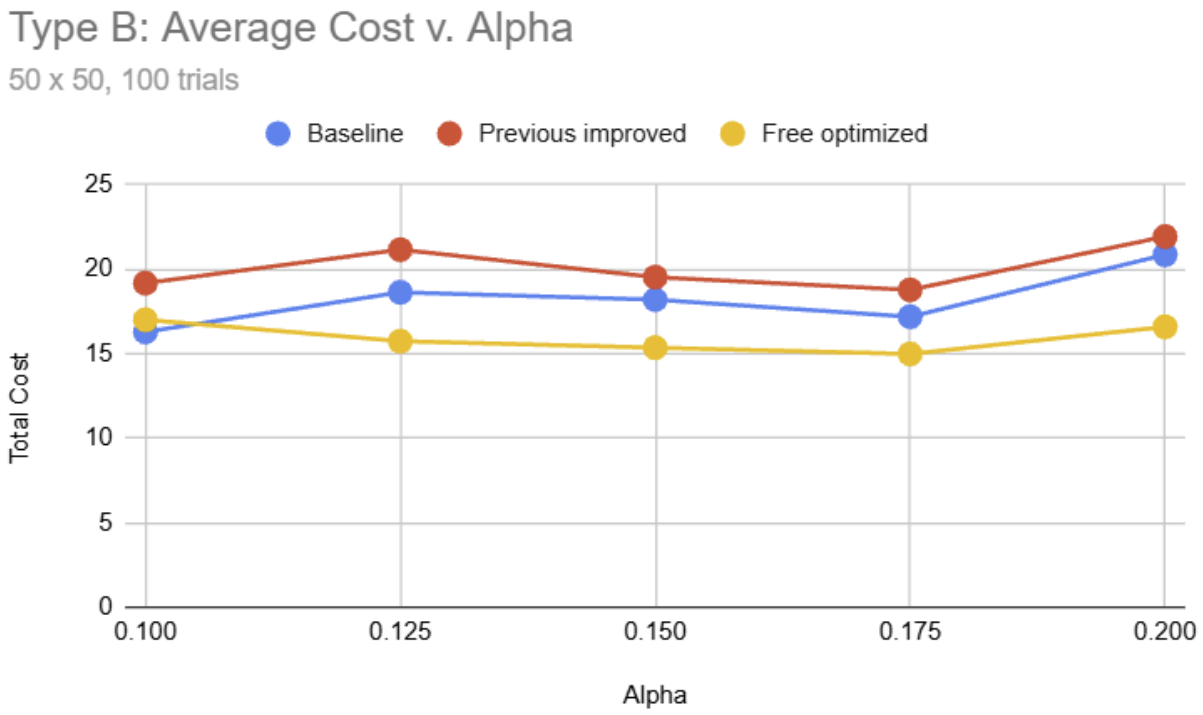
$q(a,b) = \exp[-\alpha(d(a,b) - 1)]$ for $b \neq a$ (the probability of receiving PING if the LostBot occupies another cell 'b'.)

$\mu(a) = [\sum_{b \neq a} p(b)\, q(a,b)] / [1 - f(a)]$ (the average predicted probability of PING, assuming the bot does not immediately find the LostBot.)

$V(a) = [\sum_{b \neq a} p(b)\, q(a,b)^2] / [1 - f(a)] - \mu(a)^2$ (the variance in predicted PING probabilities across the remaining possible locations.)

The final score is calculated as: $\text{score}(a) = f(a) + 4[1 - f(a)] \max(0, V(a))$

The first term rewards the probability of immediate discovery, while the second rewards positions where the possible LostBot locations predict different detector behavior. The variance is multiplied by four to normalize its maximum possible value of 0.25 to 1, placing it on the same scale as the probability of immediately finding the LostBot. This allows the strategy to consider both immediate discovery and the potential usefulness of a detector reading when selecting its next destination. When $f(a)=1$, the bot is certain that the LostBot occupies the proposed cell, so its score is set to 1 without calculating variance, which would otherwise divide by 0. The bot selects the reachable position with the highest score, resolving ties by immediate discovery probability, then walking distance, and finally randomly



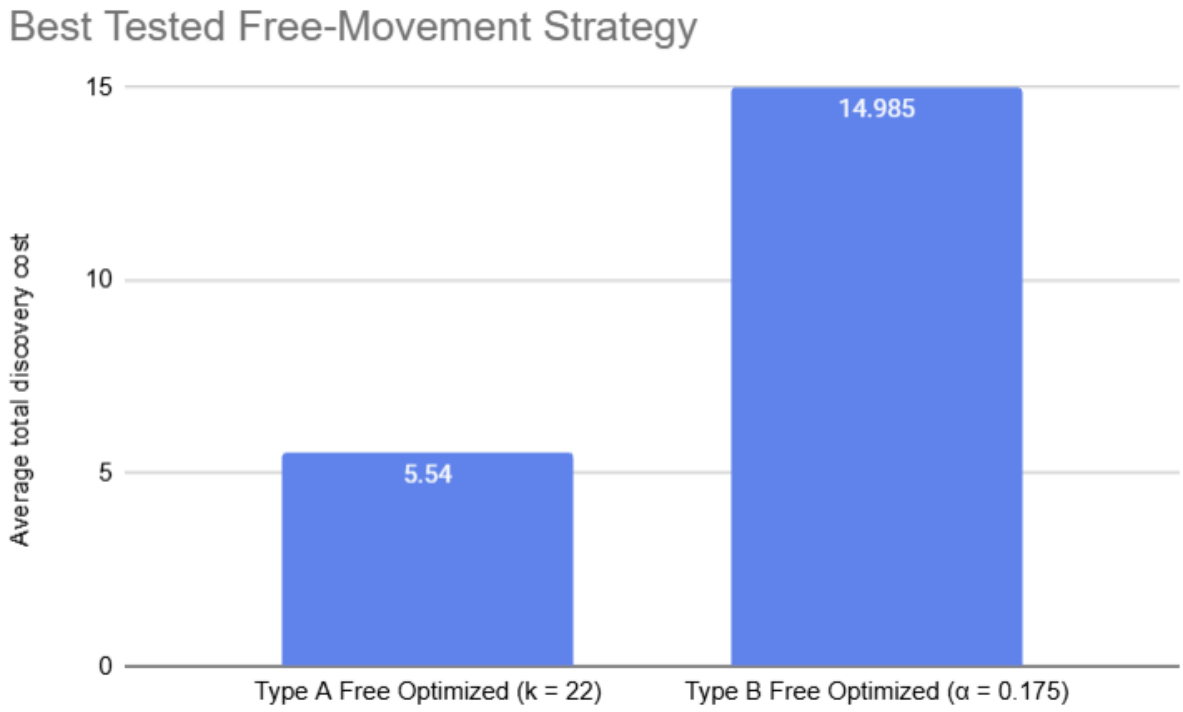
The Type B results showed a smaller improvement than Type A. The free-optimized strategy achieved its lowest average cost of 14.985 at $\alpha = 0.175$, equivalent to 29.97 detector activations. At the same α, the baseline averaged 17.185 and the previous improved strategy averaged 18.775. The new strategy therefore reduced cost by 12.8% relative to the baseline and 20.2% relative to the previous improvement. It outperformed the baseline in 57 of the 100 paired trials and the previous improved bot in 64. The average saving over the baseline was 2.200 units, with a 95% confidence interval of approximately 0.088 to 4.312. The saving over the previous improvement was 3.790 units, with a confidence interval of 1.818 to 5.762.

The baseline achieved its lowest mean cost of 16.285 at $\alpha = 0.10$, compared with the free-optimized strategy's 14.985 at $\alpha = 0.175$. This represents an observed improvement of approximately 8.0%, but the paired 95% confidence interval for the difference extends from −0.706 to 3.306 units. Since this interval includes zero, the experiment does not establish a statistically clear advantage over the independently tuned baseline, even though the free-optimized strategy performed better at several fixed α values.

Trial 42 of the final free-movement experiment (Runs 1129 and 1131) illustrates how the variance-based strategy changes Type B's decisions. At $\alpha = 0.175$, both bots began at (3, 15), with the LostBot at (23, 8), and both initially received NO_PING. The baseline selected (49, 49) for its second detection because that cell had a slightly higher probability of containing the LostBot, approximately 0.06589%. The free-optimized bot instead selected (21, 40), whose immediate discovery probability was approximately 0.06554%, but whose variance-based

information score was 0.09161 compared with the baseline's 0.03560. The free-optimized destination therefore had an information score approximately 2.57 times as large, despite its slightly lower probability of immediate discovery. Over the complete trial, the baseline required 71 detections, costing 35.5 units, while the free-optimized bot required only six detections, costing 3.0 units. Because Type B observations are stochastic, the later detector readings differed between the two bots, so the entire cost difference cannot be attributed solely to the second destination choice. Nevertheless, the initial decision demonstrates how prioritizing informative observations can produce a different search strategy from simply investigating the most probable cell.

The most efficient tested parameters in the final free-movement experiment were $k = 22$ for Type A and $\alpha = 0.175$ for Type B. However, nearby parameter values produced similar costs, and the 95% confidence intervals for their paired differences included zero. Therefore, these settings had the lowest observed averages, but the experiments do not establish them as uniquely optimal.



Type A remained substantially more efficient. At $k = 22$, Type A averaged 5.540 discovery-cost units, while Type B at $\alpha = 0.175$ averaged 14.985. Type A therefore had approximately 63.0% lower discovery cost, requiring an average of 11.08 detector activations compared with Type B's 29.97. It was cheaper in 94 of the 100 paired trials, tied twice, and performed worse four times. The average paired saving was 9.445 units, with a 95% confidence interval of approximately 8.108 to 10.782 units, providing strong evidence of Type A's

advantage under the new cost model. As with question 5, the difference can be explained by both the detector mechanisms and the strategies used to exploit them. Type A's deterministic readings allow it to eliminate entire groups of possible locations with certainty, and its free-movement strategy takes advantage of this by selecting positions that divide the remaining search space as evenly as possible. Type B's probabilistic readings generally adjust the relative probabilities of candidate locations, requiring additional observations to narrow the search. Although its variance-based strategy attempts to select more informative detection positions, it cannot guarantee that an individual reading will eliminate an entire region.

The importance of the search strategy is also apparent from the original baselines. Within the final 100-trial experiment, baseline Type A's lowest average was 43.805 at $k = 20$, while baseline Type B's lowest was 16.285 at $\alpha = 0.10$. Thus, Type B performed better when using the original baseline strategies, while Type A performed substantially better after both strategies were adapted to free movement.

Overall, Type A with $k = 22$ was the most efficient tested strategy under the free-movement cost model, achieving an average cost of 5.540 compared with Type B's 14.985 at $\alpha = 0.175$. Removing movement cost allowed Type A to prioritize the most informative detector positions without regard for travel distance, producing a substantial improvement over both earlier strategies. Type B also benefited from this change, although its improvement over the strongest independently tuned baseline was smaller and statistically less certain. These results establish Type A's advantage for the implemented and tuned strategies on the tested 50×50 ships, while recognizing that the results depend on the chosen search algorithms and experimental conditions.

Experimental data and reproducibility

All averages are taken from completed simulation records (all of which are contained in the zip). The main standard-cost comparison pools were Experiments 3, 4, and 5 (125 trials per central setting). The free-movement analysis uses the final 100-trial batch as its principal validation and cites the preceding 20- and 40-trial batches for context. Across these six main 50 × 50 batches, 8,100 recorded simulations succeeded. Standard-cost diagnostic examples are in experiment5.json, Trial 1. The report's Experiment 8 refers to the third free-movement batch, whose file is experiment3.json within the free movement run set. Trial 58 contains Type A Runs 1547 and 1548, while Trial 42 contains Type B Runs 1129 and 1131.

Spreadsheet and chart data:
https://docs.google.com/spreadsheets/d/1hbegZgqA6kGN3NpOOgbEIYmHM5hCyCMJHuVenF8LQUM/edit?usp=sharing